

```

import pandas as pd
import pandas_ta as ta
from data import fetch_candles
from config import TF_SCALP, TF_ENTRY, TF_TREND

def _indicators(df: pd.DataFrame) -> dict:
    """Compute all scalping indicators on a DataFrame and return a snapshot dict.

    c = df["Close"]
    h = df["High"]
    lo = df["Low"]
    v = df["Volume"]

    # EMAs
    df["ema9"] = ta.ema(c, length=9)
    df["ema21"] = ta.ema(c, length=21)
    df["ema50"] = ta.ema(c, length=50)

    # RSI
    df["rsi"] = ta.rsi(c, length=14)

    # MACD
    macd = ta.macd(c, fast=12, slow=26, signal=9)
    df["macd"] = macd["MACD_12_26_9"]
    df["macd_sig"] = macd["MACDs_12_26_9"]
    df["macd_hist"] = macd["MACDh_12_26_9"]

    # Bollinger Bands
    bb = ta.bbands(c, length=20, std=2)
    df["bb_u"] = bb["BBU_20_2.0"]
    df["bb_m"] = bb["BBM_20_2.0"]
    df["bb_l"] = bb["BBL_20_2.0"]

    # Stochastic
    stoch = ta.stoch(h, lo, c)
    df["stk"] = stoch["STOCHk_14_3_3"]
    df["std"] = stoch["STOCHd_14_3_3"]

    # ATR
    df["atr"] = ta.atr(h, lo, c, length=14)

    # Volume SMA
    df["vol_sma"] = ta.sma(v, length=20)

```

```

row = df.iloc[-1]
prev = df.iloc[-2]

def _ema_trend():
    if row.ema9 > row.ema21 > row.ema50: return "bullish"
    if row.ema9 < row.ema21 < row.ema50: return "bearish"
    return "mixed"

def _macd_cross():
    if prev.macd_hist < 0 and row.macd_hist > 0: return "bullish_cross"
    if prev.macd_hist > 0 and row.macd_hist < 0: return "bearish_cross"
    return "none"

def _bb_pos():
    if row.Close > row.bb_u: return "above_upper"
    if row.Close < row.bb_l: return "below_lower"
    return "inside"

def _rsi_state():
    if row.rsi > 70: return "overbought"
    if row.rsi < 30: return "oversold"
    return "neutral"

def _stoch_state():
    if row.stk > 80: return "overbought"
    if row.stk < 20: return "oversold"
    return "neutral"

return {
    "price": round(row.Close, 2),
    "ema9": round(row.ema9, 2),
    "ema21": round(row.ema21, 2),
    "ema50": round(row.ema50, 2),
    "ema_trend": _ema_trend(),
    "rsi": round(row.rsi, 1),
    "rsi_state": _rsi_state(),
    "macd": round(row.macd, 4),
    "macd_hist": round(row.macd_hist, 4),
    "macd_cross": _macd_cross(),
    "bb_upper": round(row.bb_u, 2),
    "bb_mid": round(row.bb_m, 2),
    "bb_lower": round(row.bb_l, 2),
    "bb_pos": _bb_pos(),
    "stoch_k": round(row.stk, 1),
    "stoch_d": round(row.std, 1),
    "stoch_state": _stoch_state(),

```

```

    "atr":      round(row.atr, 2),
    "volume":   int(row.Volume),
    "vol_avg":  int(row.vol_sma),
    "vol_spike": row.Volume > row.vol_sma * 1.5,
    "last_5": [
        {"o": round(df.iloc[i].Open, 2),
         "h": round(df.iloc[i].High, 2),
         "l": round(df.iloc[i].Low, 2),
         "c": round(df.iloc[i].Close, 2)}
        for i in range(-5, 0)
    ],
}

```

```

def full_analysis() -> dict:
    """Return indicator snapshot for all three timeframes."""
    return {
        TF_SCALP: _indicators(fetch_candles(TF_SCALP, "1d")),
        TF_ENTRY: _indicators(fetch_candles(TF_ENTRY, "5d")),
        TF_TREND: _indicators(fetch_candles(TF_TREND, "5d")),
    }

```